task2

任务完成

- 完成了 EnCodec 章节的环境配置与依赖安装

  “Pasted image 20260418001752.png” could not be found.

  ```
  ✔ JSON API cask.jws.json
  ✔ JSON API formula.jws.json
  ==> Fetching downloads for: jq
  Warning: Bottle missing, falling back to the default domain...
  ==> Downloading https://ghcr.io/v2/homebrew/core/oniguruma/manifests/6.9.10
  Warning: Bottle missing, falling back to the default domain...
  ==> Downloading https://ghcr.io/v2/homebrew/core/jq/manifests/1.8.1
  ✔ Bottle oniguruma (6.9.10)
  ✔ Bottle jq (1.8.1)
  ==> Installing jq dependency: oniguruma
  ==> Pouring oniguruma--6.9.10.arm64_tahoe.bottle.tar.gz
  🍺  /opt/homebrew/Cellar/oniguruma/6.9.10: 15 files, 1.5MB
  ==> Pouring jq--1.8.1.arm64_tahoe.bottle.tar.gz
  🍺  /opt/homebrew/Cellar/jq/1.8.1: 20 files, 1.2MB
  ==> Running `brew cleanup jq`...
  Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
  Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
  ```

- 验证了深度学习框架环境
  确认了 PyTorch、torchaudio、encodec 可正常导入，并检查了设备可用性（CPU / CUDA / MPS）
- 完成了缓存与镜像源配置
  设置模型缓存路径与 HF 镜像地址，降低模型下载失败概率
- 学习并实践了 MAESTRO 数据集下载流程
  理解了批量下载中断后的续传思路：定位中断文件后继续顺序下载

```python
import os
import torch
import librosa

AUDIO_DIR = "/Users/ye/Downloads/maestro_2004_wav/2004"
TARGET_SR = 24000

audio_tensors = []

for fname in os.listdir(AUDIO_DIR):
    if fname.endswith(".wav"):
        path = os.path.join(AUDIO_DIR, fname)

        # librosa 直接解 mp3 (不依赖 torchcodec)
        wav, sr = librosa.load(path, sr=TARGET_SR, mono=True)

        # 转成 torch tensor: [1, T]
        wav = torch.from_numpy(wav).unsqueeze(0)

        audio_tensors.append(wav)

print(f"Loaded {len(audio_tensors)} audios.")

for i, wav in enumerate(audio_tensors[:5]):
    duration = wav.shape[1] / TARGET_SR
    print(f"Audio {i+1}: shape={wav.shape}, duration={duration:.2f}s")
```

[13] ✓ 29.7s Python

```
/Users/ye/miniconda3/envs/musiclm/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not
  from .autonotebook import tqdm as notebook_tqdm
Loaded 71 audios.
Audio 1: shape=torch.Size([1, 7287381]), duration=303.64s
Audio 2: shape=torch.Size([1, 33702805]), duration=1404.28s
Audio 3: shape=torch.Size([1, 6900724]), duration=287.53s
Audio 4: shape=torch.Size([1, 6418527]), duration=267.44s
Audio 5: shape=torch.Size([1, 23891771]), duration=995.49s
```

- 完成了音频预处理流程（加载 + 重采样）
  使用 librosa.load 将音频统一到 24kHz，并转换为模型可用张量格式
- 对比了 torchaudio.load 的加载方案
  实现了双声道转单声道与统一采样率处理，理解了为何训练前要统一输入分布
- 学习了 EnCodec 量化器 RVQ 的核心思想
  RVQ 是逐层残差量化：上一层保留主要信息，下一层补充细节，最终得到更紧凑的离散表示

```python
from encodec import EncodecModel

model = EncodecModel.encodec_model_24khz()
quantizer = model.quantizer

codebooks = [layer.codebook for layer in quantizer.vq.layers]
print([cb.shape for cb in codebooks])
```

[17] ✓ 21.9s Python

/Users/ye/miniconda3/envs/musiclm/lib/python3.11/site-packages/torch/nn/utils/weight_norm.py:144: FutureWarning: `torch.nn.utils.weight_norm` is deprecated in favor of `torch.nn.utils.parametrizations.weight_norm`.
WeightNorm.apply(module, name, dim)

Downloading: "https://dl.fbaipublicfiles.com/encodec/v0/encodec_24khz-d7cc33bc.th" to /Users/ye/.cache/torch/hub/checkpoints/encodec_24khz-d7cc33bc.th
100%|[redacted]| 88.9M/88.9M [00:20<00:00, 4.49MB/s]torch.Size([1024, 128]), torch.Size([1024, 128])]

- 查看了 codebook 结构与层间差异
  观察到每层 codebook 的向量分布不同，进一步理解多级量化器的作用分工

```python
from encodec import EncodecModel

model = EncodecModel.encodec_model_24khz()
quantizer = model.quantizer

layer = quantizer.vq.layers[0]   # 先看第 1 个 RVQ layer
codebook = layer.codebook        # shape: (num_codes, 128)

print(codebook[:1])              # 看前 1 个 128 维向量
```

[18] ✓ 0.1s Python

```
tensor([[  5.3395,  13.1336,  -3.3514,  -0.5637,   3.2171,  11.4735,  -8.4687,
           0.4034,   0.7212,   8.6175,   5.7454,  -1.2644,   2.0950, -10.3753,
          -0.2346, -12.8185,  -6.0519,   6.8965,  -0.8932,   0.3201, -12.9295,
          -4.5874,   1.0253,  -3.0017,   6.2650,   1.6609,  13.7952,  -4.4926,
          -9.5459,  -2.4131,  -9.9515,   0.5068,   2.3795,  -8.9276,   2.6603,
          -4.5152,   2.6780, -11.3535,  -1.2518,   4.9170,   3.6355,   1.2266,
          18.9035,   5.1384,   1.8919,  -1.1213,  -4.3447,   6.0878,   1.4327,
          -4.1681,  -8.1615,  -8.1800,   0.3895,   5.5438,  -9.7784,   2.4889,
          -2.7340,  -8.0559,  -4.9292,  -0.4302, -11.3153,  -6.0190,   0.1818,
          -3.4706, -11.1346,  -4.3844,   2.1268,  -2.7332,   0.0789,  -0.6986,
          -3.0472,  -6.6059,  -1.3161,   1.0986,   0.2661, -11.3157,  -3.3526,
          -1.5227,  -0.0636,  -2.2600,  -1.9390,  -7.0191, -10.3219,   7.9803,
           5.6771,  -1.0727,   8.7551,  -7.5301,   0.9749,   0.5898,  12.7960,
           3.7519,  -0.7389,   6.8869, -11.4781,   7.0113,   3.0283,  20.7458,
          -1.8931,   6.5462,   7.0486,  13.2628,   5.7293,   1.9618, -12.3246,
          -3.3307,  -0.7211,  -6.1105,   1.1360,   3.0271, -13.1079,  -5.0016,
          -6.7679,   0.6489,  -4.9507,  -1.6126,   8.9500,  -0.5464,   4.2155,
          -4.8056,  -0.6793,   7.9327,   5.2310,   0.3282,   2.5067,   2.2543,
          -4.5506,   3.7425]])
```

```python
for l, layer in enumerate(quantizer.vq.layers):
    cb = layer.codebook
    print(f"第 {l} 级量化器: shape={cb.shape}")
    print(f"取这级 codebook 中的第一个向量: {cb[0][:8]}...直到 128 维"+"\n")   # 每层第一个向量的前 8 维
```

[19] ✓ 0.0s Python

```
第 0 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([ 5.3395, 13.1336, -3.3514, -0.5637,  3.2171, 11.4735, -8.4687,  0.403

第 1 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([-0.5989, -0.5179,  0.0062,  0.3614,  0.4851,  0.4453,  0.7357,  0.365

第 2 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([-0.1872, -0.2495, -0.5744, -0.0337,  0.9889, -0.1247,  0.3137, -0.310

第 3 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([-1.0626, -0.9499,  1.0589,  0.2603,  0.3291, -0.0505, -0.5498,  0.789

第 4 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([-0.7457, -0.3382,  0.6972,  0.2706, -0.1005, -0.2790, -0.3204,  0.519

第 5 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([-0.2221, -0.6490,  0.4214, -0.0472,  0.4390, -0.6089, -0.6238,  0.244

第 6 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([-0.2610,  0.1445,  0.1345, -0.0533, -0.2809,  0.0950,  0.1060,  0.280

第 7 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([-0.1706,  0.4021,  0.4553, -0.2190,  0.2707, -0.1749, -0.1064, -0.070

第 8 级量化器: shape=torch.Size([1024, 128])
...

第 31 级量化器: shape=torch.Size([1024, 128])
取这级 codebook 中的第一个向量: tensor([ 3.2981, -0.2873,  0.8708,  1.2206,  0.6484, -1.2183,  0.2486,  0.956
```

- 完成了音频分块编码为 token 的主流程
  将音频按 10 秒切块，执行 encode，保存每首音乐对应的 token 文件
  输出：Using device: cpu Audio 1/71

/Users/ye/miniconda3/envs/musiclm/lib/python3.11/site-packages/torch/nn/utils/weight_norm.py:144: FutureWarning: `torch.nn.utils.weight_norm` is deprecated in favor of `torch.nn.utils.parametrizations.weight_norm`. WeightNorm.apply(module, name, dim)

codes shape: torch.Size([4, 750]) Saved maestro_tokens_3kbps/music_0000.pt Audio 2/71
codes shape: torch.Size([4, 750]) Saved maestro_tokens_3kbps/music_0001.pt Audio 3/71
codes shape: torch.Size([4, 750]) Saved maestro_tokens_3kbps/music_0002.pt Audio 4/71
codes shape: torch.Size([4, 750]) Saved maestro_tokens_3kbps/music_0003.pt Audio 5/71
codes shape: torch.Size([4, 750]) Saved maestro_tokens_3kbps/music_0004.pt Audio 6/71
codes shape: torch.Size([4, 750]) Saved maestro_tokens_3kbps/music_0005.pt Audio 7/71
codes shape: torch.Size([4, 750])

...

codes shape: torch.Size([4, 750]) Saved maestro_tokens_3kbps/music_0016.pt Audio 71/71

- 完成了索引文件保存
  保存数据集元信息（样本数、带宽、采样率、chunk 大小），便于后续训练直接读取

5、保存 index

```python
import json

index = {
    "num_music": len(audio_tensors),
    "bandwidth": 3.0,
    "sample_rate": 24000,
    "chunk_size": CHUNK_SIZE,
    "music": [
        {"id": i, "file": f"music_{i:04d}.pt"}
        for i in range(len(audio_tensors))
    ]
}

with open("maestro_tokens_3kbps/index.json", "w") as f:
    json.dump(index, f, indent=2)
```

[27] ✓ 0.0s Python

学习心得

本次学习把我对“音频如何进入生成模型”的理解从概念层推进到了工程层。之前更关注频谱和特征表示，这次重点是把连续波形压缩成可训练的离散 token，真正接触到了 Audio Tokenizer 在 Music Transformer 流程中的入口作用。
我印象最深的是两点：第一，预处理的一致性（采样率、声道、切块策略）直接决定了后续训练稳定性；第二，RVQ 并不是一次性量化，而是分层逼近，体现了压缩率和保真度之间的平衡。
后续我会继续关注 token 质量与生成效果之间的关系，并尝试不同带宽参数对结果的影响。